LIVER PATIENT PREDICTION MODEL

CAPSTONE PROJECT IN FULLFILLMENT OF REQUIREMENT FOR CERTIFICATION AS CERTIFIED DATA SCIENTISTS AT DATAMITES GLOBAL TRAINING INSTITUTE

Authors:

Sn	Name	Email
1.	Sindhya Sivaram	cynthiasunil8@gmail.com
2.	Vinil Sukumar	vinilsukumar2698@gmail.com
3.	Keerthi Cheerath	keerthicheerath@gmail.com
4.	Musa Leteipa Sere	musa.sere7@gmail.com

PRELIMINARY SECTION

PROJECT OVERVIEW

This project seeks to develop a machine learning model to help predict patients with liver disease. Patients with Liver disease have been continuously increasing because of excessive consumption of alcohol, inhale of harmful gases, intake of contaminated food, pickles and drugs. The model will reduce the burden on doctors by classifying patients as 'liver patient' or 'not liver patient'.

The dataset used in this project has been provided by Datamites Training Institute, India. The dataset contains 416 liver patient records and 167 non liver patient records collected from North East of Andhra Pradesh, India. The "Target" column is a class label used to divide groups into liver patients (liver disease) or not (no disease). This data set contains 441 male patient records and 142 female patient records. Any patient whose age exceeded 89 is listed as being of age "90".

DOMAIN ANALYSIS

The domain for this is project is healthcare. The attributes from dataset underconsideration are discussed below:

- **Age of the patient**: Age is considered a significant factor in a medical process. Some diseases are age related (individuals of certain age range may be prone to certain diseases) and also age is an important consideration when prescribing effective

medication so as to avoid over-dose or under-dose. Some tests have varying normal range across with to age.

- **Gender of the patient**: Gender affects biological factors, lifestyle choices and social-cultural expectations. These impact on the patients attitude and experiences with the healthcare system. Normal ranges for some tests differ for males and females.

- **Total Bilirubin**: Bilirubin is a substance produced when the red blood cell breakdown. The liver removes this substance from the blood stream. Higher level of Total Bilirubin in the blood indicate liver problems, anemia, gallbladder problems, etc. The total bilirubin combines direct and indirect bilirubin. The total bilirubin levels of 0.1 to 1.2 mg/dL is considere normal range for an adult.

- **Direct Bilirubin:** Direct bilirubin levels of 0.1-0.3 mg/dL is considered normal range for a healthy adult person.

- **Alkaline Phosphotase (ALP):** Alkaline phosphotase is found in all body tissue. The ALP test is considered important in checking the health of liver and bones. A normal level range of ALP in the blood test for adult male is 40 to 130 U/L while for an adult female is 35 to 104 U/L. For liver tissues, a normal range of ALP for adult is 15.8 to 71.9 U/L. High level indicate issues with liver.

- **Alamine Aminotransferase (ALT)**: ALT is considered a significant predictor of liver health and other medical conditions. Normal levels of ALT for male is 7-55 U/L and 7-45 U/L for females. High levels indicate damage to liver

- **Aspartate Aminotransferase (AST)**: AST is a blood test that measures the levels of AST enzymes in the body. Common accepted ranges are approximately 8-43 U/L for female adult and 8-48 U/L for male adults. High level of AST indicates problems with liver, heart and muscles as well as other medical conditions like hepatitis, cirrhosis, etc. High levels indicate damage to liver, muscle or heart issues. Low levels indicate vitamin B6 deficiency, kidney issues, certain inflammatory conditions.

- **Total Protiens:** Total protient measures combination of albumin and globulins levels in the blood. 6.0 to 8.3 g/dL levels of total protein are considered normal range for a healthy adult person. Low levels indicate malnutrition, liver issues, kidney issues, etc. High levels indicate dehydration, inflammation, etc.

- **Albumin**: Albumin is a protein produced by the liver. Albumin test measures the amount of albumin in the blood or urine. 3.4 to 5.4 g/dL level of albumin is considered okay. Lower levels indicates liver or kidney issues while higher level indicate dehydration.

- **Albumin and Globulin Ratio (A/G ratio):** 1.0 to 2.0 A/G ratio is considered a normal range for a healthy adult. A ratio below 1 indicates issues with the liver, kidneys, or inflamation. High ratio indicate dehydration.

- **Target:** The target attribute splits the data into two sets: 1 for patient with liver disease and 2 for patient with no liver disease disease

TASKS:

The team undertook the following tasks in an effort to build a good model:

- Task 1:-Prepare a complete data analysis report on the given data.
- Task 2:-Create a predictive model with implementation of different classifiers on liver patient diseases dataset to predict liver diseases.
- Task 3:- Create an analysis to show on what basis you have designed your model.

PREPROCESSING SECTION

Preprocessing Libraries:

The following libraries will be used in data preprocessing and cleansing:

- Numpy for numerical computing,
- Pandas for dataframes handling,
- Maltplotlib for Visualization Techniques,
- Seaborn for drawing heatmaps to find correlations,
- LabelEncoder for tranforming categorical columns into numerical
- Warnings for Ignoring the warning templates

In [1]:
```
# Libraries for data framing, data manipulation and computing
import numpy as np
import pandas as pd

# Libraries for data visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Library for suppressing warnings
import warnings
warnings.filterwarnings("ignore")
```

Loading the Dataset:

In [2]:
```
## Loading the Dataset
df = pd.read_csv("Indian Liver Patient Dataset (ILPD).csv",header=None)
```

Basic Checks

Basic checks are perform to understand the data as well as identify any direct issue that require handling

In [3]:
```
# checking head and tail of the data
df.head(), df.tail()
```

Out[3]:
```
(   0       1     2    3    4   5    6    7    8     9   10
 0  65  Female   0.7  0.1  187  16   18  6.8  3.3  0.90   1
 1  62    Male  10.9  5.5  699  64  100  7.5  3.2  0.74   1
 2  62    Male   7.3  4.1  490  60   68  7.0  3.3  0.89   1
 3  58    Male   1.0  0.4  182  14   20  6.8  3.4  1.00   1
 4  72    Male   3.9  2.0  195  27   59  7.3  2.4  0.40   1,
      0     1    2    3    4   5   6    7    8     9   10
 578  60  Male  0.5  0.1  500  20  34  5.9  1.6  0.37   2
 579  40  Male  0.6  0.1   98  35  31  6.0  3.2  1.10   1
 580  52  Male  0.8  0.2  245  48  49  6.4  3.2  1.00   1
 581  31  Male  1.3  0.5  184  29  32  6.8  3.4  1.00   1
 582  38  Male  1.0  0.3  216  21  24  7.3  4.4  1.50   2)
```

In [4]:
```
# checking the columns names
df.columns
```

Out[4]:
```
Index([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10], dtype='int64')
```

Observations:

- Dataset has no header

In [5]:
```
# Renaming columns/attributes
df.columns = [
    'Age',
    'Gender',
    'Total_Bilirubin',
    'Direct_Bilirubin',
    'Alkaline_Phosphotase',
    'Alamine_Aminotransferase',
    'Aspartate_Aminotransferase',
    'Total_Protiens',
    'Albumin',
    'Albumin_and_Globulin_Ratio',
    'Target'
]
```

In [6]:
```
# Checking column names after renaming
df.columns
```

Out[6]:
```
Index(['Age', 'Gender', 'Total_Bilirubin', 'Direct_Bilirubin',
       'Alkaline_Phosphotase', 'Alamine_Aminotransferase',
       'Aspartate_Aminotransferase', 'Total_Protiens', 'Albumin',
       'Albumin_and_Globulin_Ratio', 'Target'],
      dtype='object')
```

In [7]:
```
# Checking details about dataframe like total number of columns,column names and da
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 583 entries, 0 to 582
Data columns (total 11 columns):
 #   Column                      Non-Null Count  Dtype  
---  ------                      --------------  -----  
 0   Age                         583 non-null    int64  
 1   Gender                      583 non-null    object 
 2   Total_Bilirubin             583 non-null    float64
 3   Direct_Bilirubin            583 non-null    float64
 4   Alkaline_Phosphotase        583 non-null    int64  
 5   Alamine_Aminotransferase    583 non-null    int64  
 6   Aspartate_Aminotransferase  583 non-null    int64  
 7   Total_Protiens              583 non-null    float64
 8   Albumin                     583 non-null    float64
 9   Albumin_and_Globulin_Ratio  579 non-null    float64
 10  Target                      583 non-null    int64  
dtypes: float64(5), int64(5), object(1)
memory usage: 50.2+ KB
```

Observations:

- There are 10 numerical columns and one categorical column
- Gender column require encoding conversion. To be done during data cleansing
- Albumin_Globulin_Rayio column has four null values

In [8]:
```
# check uniques values for each column
df.nunique()
```

Out[8]:
```
Age                           72
Gender                         2
Total_Bilirubin              113
Direct_Bilirubin              80
Alkaline_Phosphotase         263
Alamine_Aminotransferase     152
Aspartate_Aminotransferase   177
Total_Protiens                58
Albumin                       40
Albumin_and_Globulin_Ratio    69
Target                         2
dtype: int64
```

Observation:

- gender and target columns have 2 unique values. It can be confirm that these two are caterical columns though target has already been encoded

In [9]:
```
# Checking information about toal number of rowns and columnns in the dataframe
df.shape
```

Out[9]:
```
(583, 11)
```

In [10]:
```
# Checking Summary Statistics for the numerical columns
df.describe()
```

Out[10]:

	Age	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotra
count	583.000000	583.000000	583.000000	583.000000	58
mean	44.746141	3.298799	1.486106	290.576329	8
std	16.189833	6.209522	2.808498	242.937989	18
min	4.000000	0.400000	0.100000	63.000000	1
25%	33.000000	0.800000	0.200000	175.500000	2
50%	45.000000	1.000000	0.300000	208.000000	3
75%	58.000000	2.600000	1.300000	298.000000	6
max	90.000000	75.000000	19.700000	2110.000000	200

In [11]:
```
# Checking Summary Statistics for categorical columns
df.describe(include ='object')
```

Out[11]:

	Gender
count	583
unique	2
top	Male
freq	441

Observations:

- Dataframe has no constant or unique columns
- Greater variability observed in columns including age, total bilirubin, etc. Feature scaling should be considered
- No columns with zero minimum as this is not expect in alive human being

In [12]:
```
# Checking duplicate records
df.duplicated().sum()
```

Out[12]: np.int64(13)

Observations:

- There are 13 duplicated records. To be dropped during data cleansing

In [13]:
```
# Checking for null values
df.isnull().sum()
```

Out[13]:
```
Age                          0
Gender                       0
Total_Bilirubin              0
Direct_Bilirubin             0
Alkaline_Phosphotase         0
Alamine_Aminotransferase     0
Aspartate_Aminotransferase   0
Total_Protiens               0
Albumin                      0
Albumin_and_Globulin_Ratio   4
Target                       0
dtype: int64
```

Observations:

- There are 4 null values found in Albumin_Globulin_ratio column.

In [14]:
```
# Display records with missing value for more observation
df[df.isnull().any(axis=1)]
```

Out[14]:

	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotr
209	45	Female	0.9	0.3	189	
241	51	Male	0.8	0.2	230	
253	35	Female	0.6	0.2	180	
312	27	Male	1.3	0.6	106	

Observations:

- No noticeable pattern found in the records with missing values. It is assumed the missing values are Missing Completely At Random (MCAR) hence will be filled using mean of the column

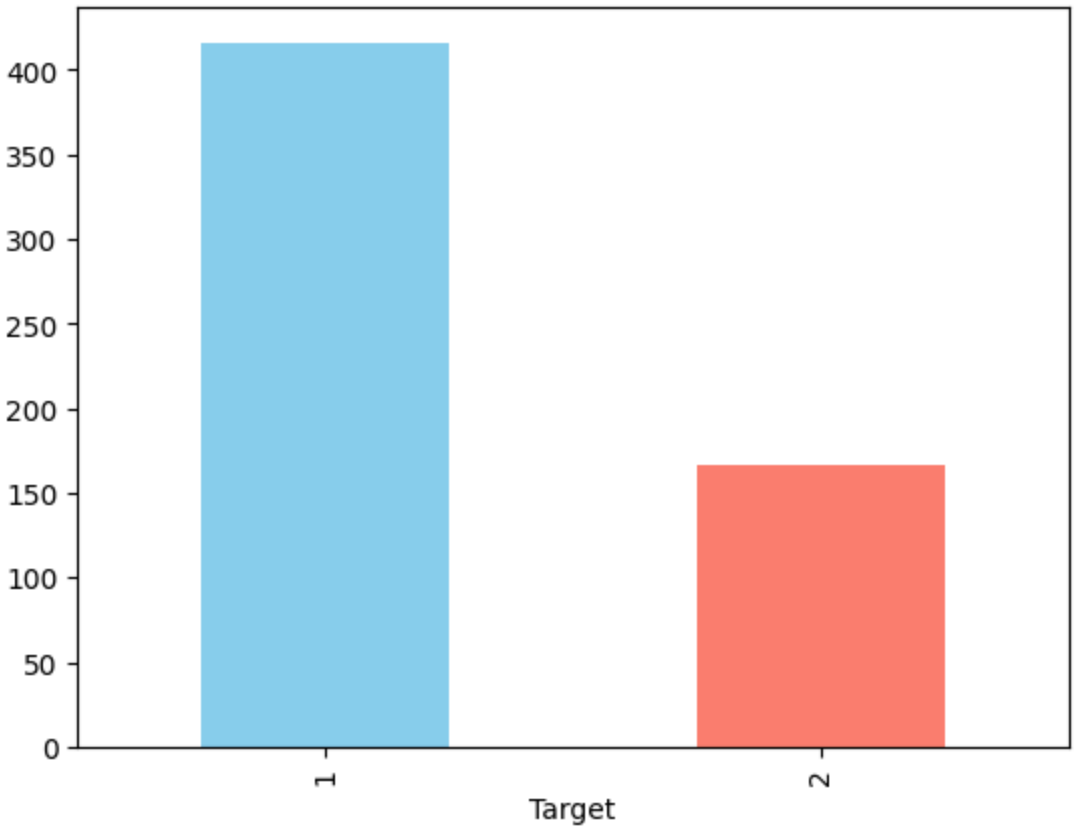
Exploratory Data Analysis (EDA)

EDA is performed to get more understanding about the data distribution and identify any useful patterns. Three category of analyses will be undertaken in this sub-section: Univariate, bivariate and multivariate analyses

Univariate Analysis

In [15]:
```
# Visualizing target column using bar plot
tgt_count=df['Target'].value_counts()
tgt_count.plot(kind='bar',color=['skyblue','salmon'])
```

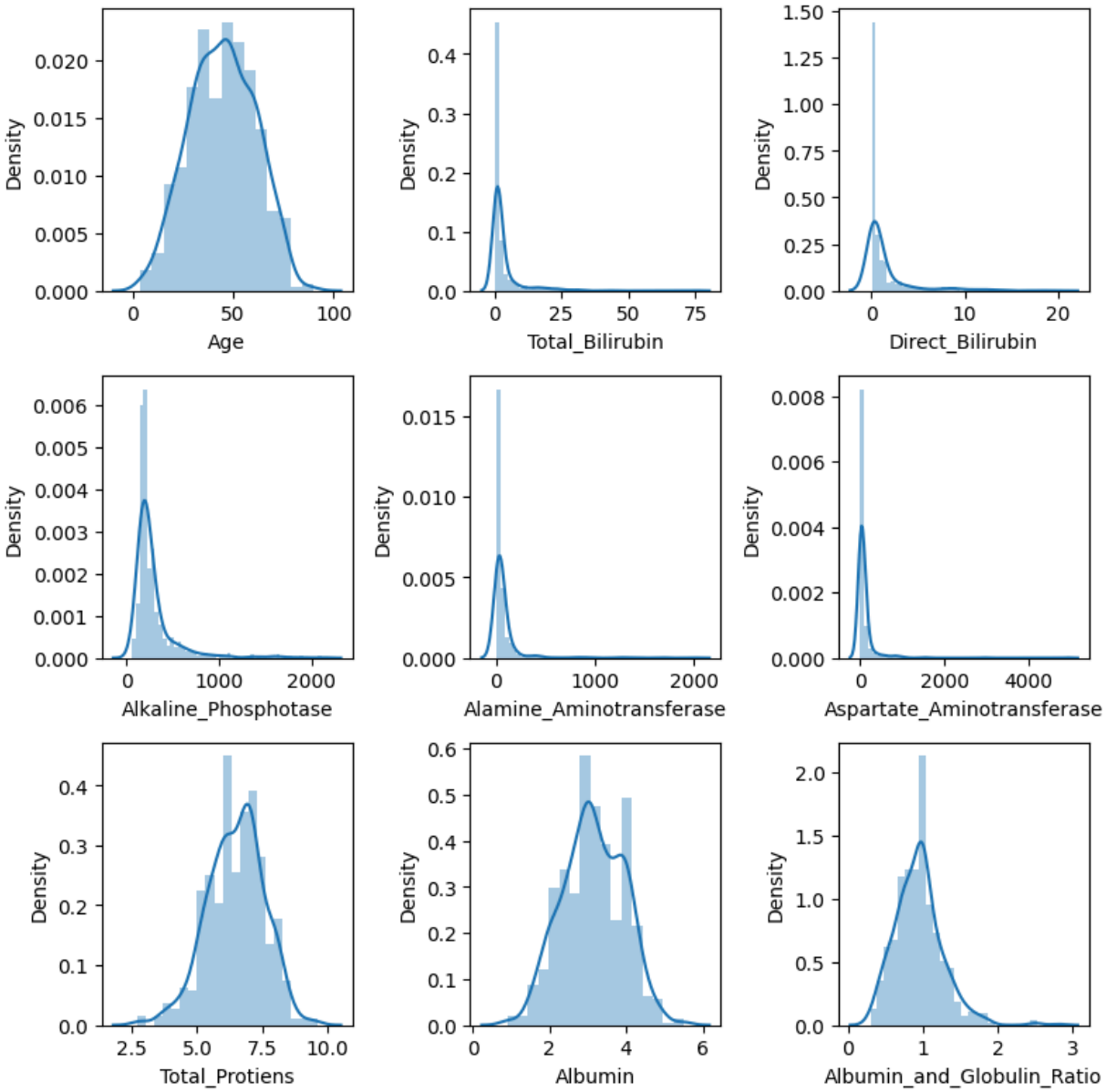
Out[15]:
```
<Axes: xlabel='Target'>
```



Observations:

- class 1 occurances are more. Since the target is binary, balancing the classes will help the model train well.

In [16]:
```
#Visualisation of each numerical columns using distribution plots
plt.figure(figsize=(8,8))
plot = 1

for i in df.drop('Gender',axis=1):
    if plot<=9:
        ax = plt.subplot(3,3,plot)
        sns.distplot(x=df[i])
        plt.xlabel(i)
    plot+=1
plt.tight_layout()

#Insights
```

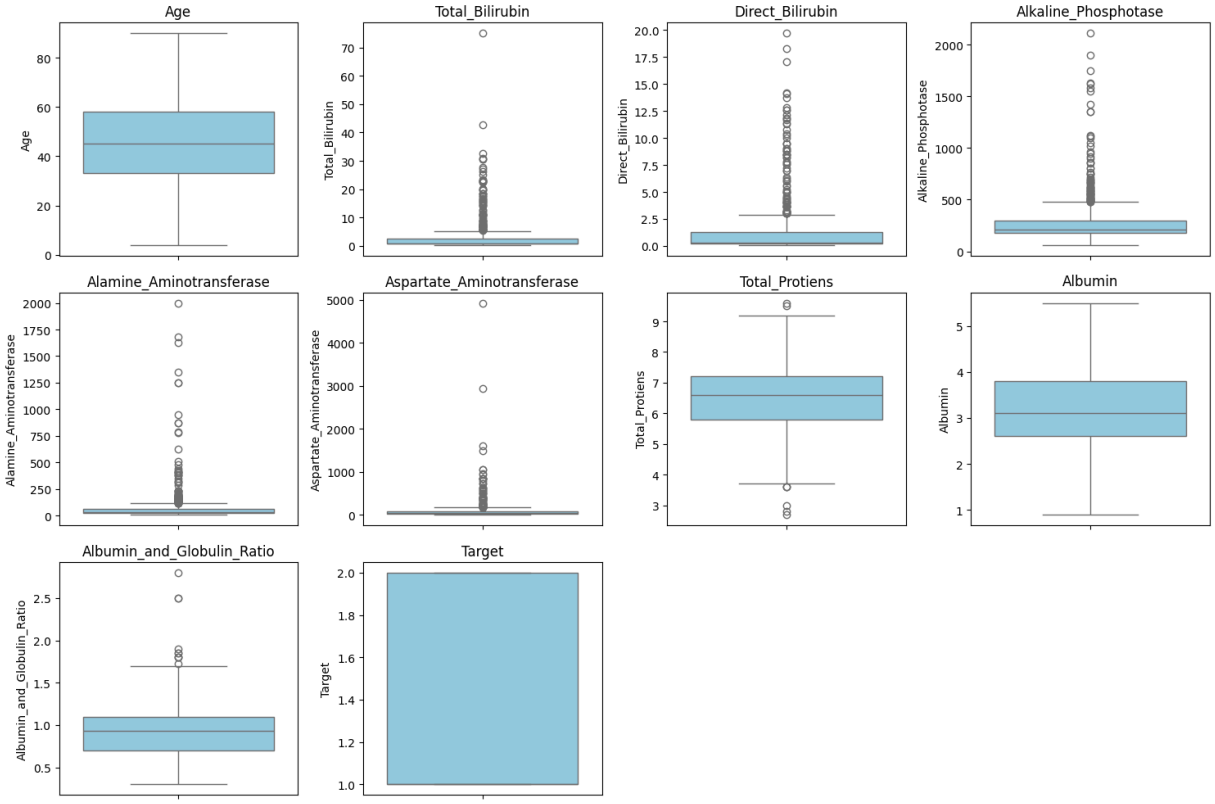


Observations:

- Total Bilirubin: Often right-skewed (many patients have low bilirubin, but some have very high values).
- Direct Bilirubin: Similar right skew.
- Alkaline Phosphotase: Typically has a wide spread with outliers.
- Albumin: More normally distributed but shifted towards lower side in patients.
- Albumin/Globulin Ratio: Often left-skewed or bimodal.
- Age: Approximates normal distribution but centered around middle-age.

In [17]:
```
# Visulisation of each numerical column using boxplot
# Select only numerical columns
num_cols = df.select_dtypes(include=['float64', 'int64']).columns [:]

# Plot boxplots for all numerical columns
plt.figure(figsize=(15, 10))
for i, col in enumerate(num_cols, 1):
```

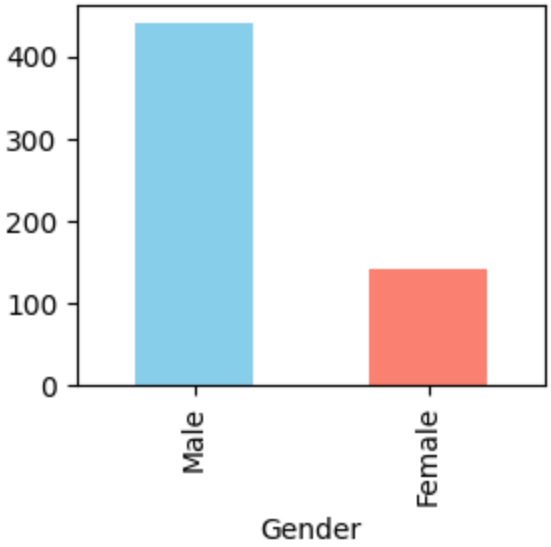
```
    plt.subplot(3, 4, i)
    sns.boxplot(y=df[col], color="skyblue")
    plt.title(col)

plt.tight_layout()
plt.show()
```



Observations:

- Age values range roughly between 20–90. No extreme outliers; distribution is fairly even.
- Total Bilirubin & Direct Bilirubin: Strong right-skewed with several outliers.Indicates some patients have abnormally high bilirubin levels → a key indicator of liver disease.
- Alkaline Phosphotase (ALP): Very wide range with many extreme outliers. Suggests significant variation in enzyme levels, important for diagnosis.
- Alamine Aminotransferase (ALT) & Aspartate Aminotransferase (AST): Both show extreme outliers with right-skewed distributions. High enzyme values are common in liver patients → consistent with medical knowledge.
- Total Proteins, Albumin, Albumin/Globulin Ratio: Relatively less skewed.
- Some outliers in Albumin/Globulin ratio → could be associated with abnormal liver function.

In [18]:
```
# Visualisation of categorial column using barplot
plt.figure(figsize=(3,3))
gender_count=df['Gender'].value_counts()
gender_count.plot(kind='bar',color=['skyblue','salmon'])
plt.tight_layout()
```

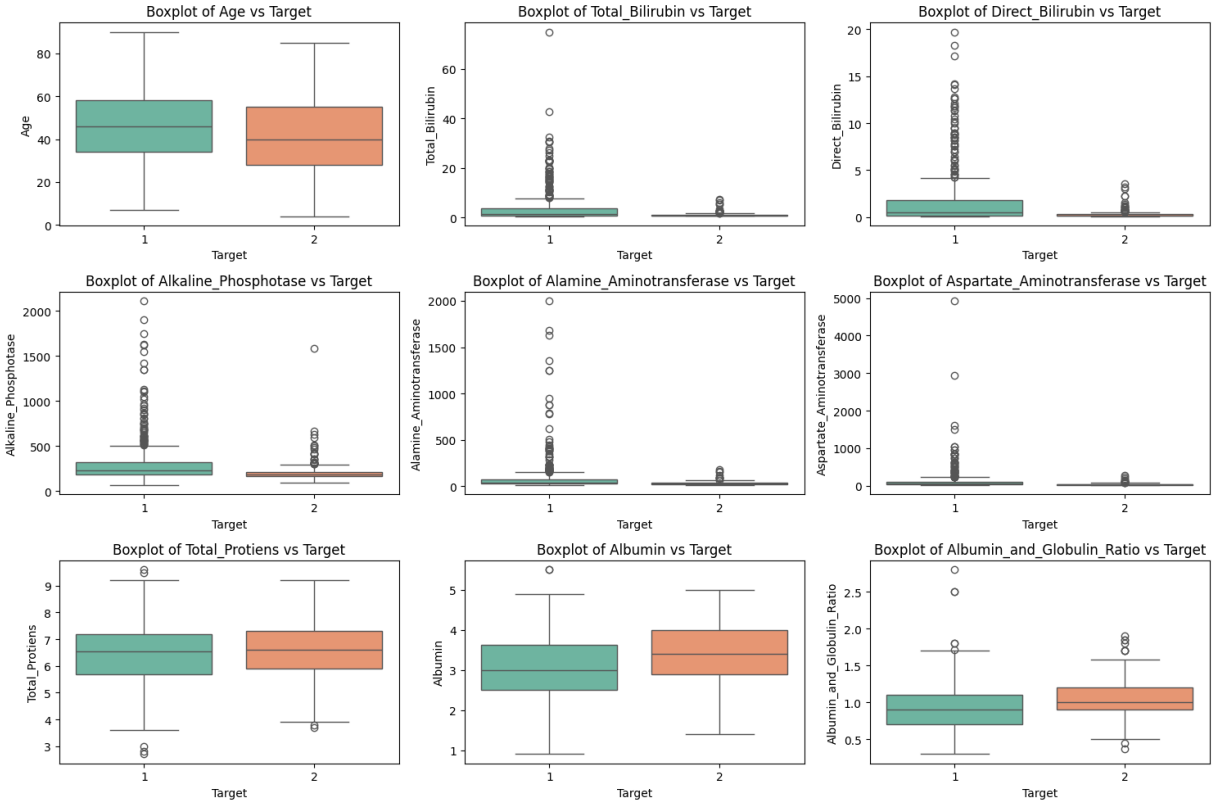


Observations:

- Male are more than the female.

BIVARIATE ANALYSIS

In [19]:
```
# Visualising each input column against the target Column Using Boxplot

# Ensure numerical columns only
num_cols = df.select_dtypes(include=['int64','float64']).columns
num_cols = [col for col in num_cols if col != 'Target']  # exclude target

# Plot boxplots for each numerical column vs Target
plt.figure(figsize=(15, 10))
for i, col in enumerate(num_cols, 1):
    plt.subplot((len(num_cols)+2)//3, 3, i)
    sns.boxplot(x="Target", y=col, data=df, palette="Set2")
    plt.title(f"Boxplot of {col} vs Target")

plt.tight_layout()
plt.show()
```

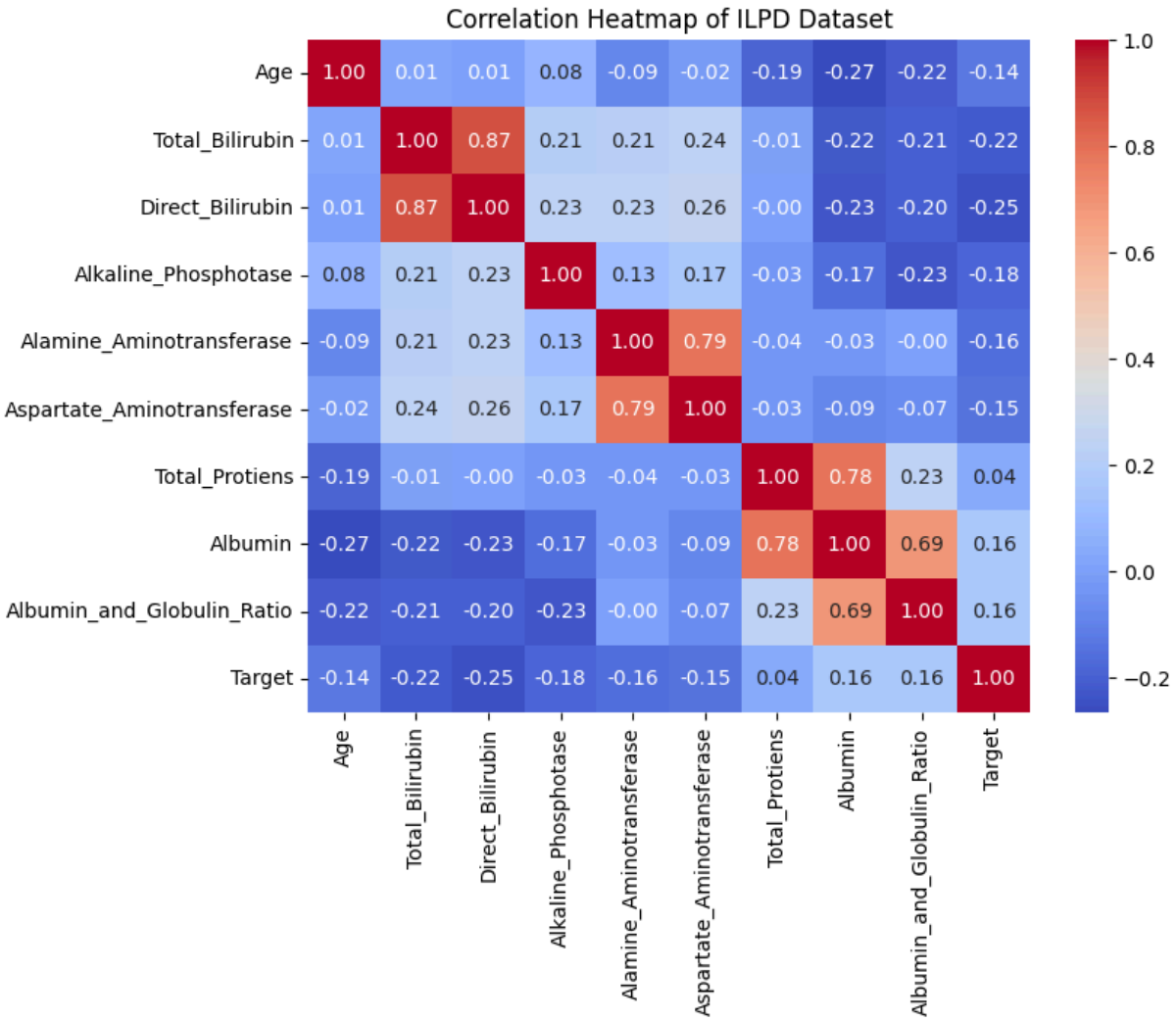

Boxplot of Age vs Target
Age
Target
Boxplot of Total_Bilirubin vs Target
Total_Bilirubin
Target
Boxplot of Direct_Bilirubin vs Target
Direct_Bilirubin
Target
Boxplot of Alkaline_Phosphotase vs Target
Alkaline_Phosphotase
Target
Boxplot of Alamine_Aminotransferase vs Target
Alamine_Aminotransferase
Target
Boxplot of Aspartate_Aminotransferase vs Target
Aspartate_Aminotransferase
Target
Boxplot of Total_Protiens vs Target
Total_Protiens
Target
Boxplot of Albumin vs Target
Albumin
Target
Boxplot of Albumin_and_Globulin_Ratio vs Target
Albumin_and_Globulin_Ratio
Target

Observations:

- Age vs Target:
  - Patients with liver disease (Target=1) are often in slightly older age groups compared to non-patients.
  - Wider spread in diseased group.
- Total Bilirubin (TB) & Direct Bilirubin (DB) vs Target:
  - Median bilirubin levels are much higher for liver disease patients.
  - Outliers in non-patients exist but are fewer.
- Alkaline Phosphatase (Alkphos) vs Target:
  - Diseased patients show higher values and greater spread.
  - Useful discriminating feature.
- Total Protein (TP), Albumin (ALB), and A/G ratio vs Target:
  - Liver disease patients usually have lower albumin and A/G ratio.
  - TP shows overlapping distribution, less discriminative.
- **Conclusion**: From bivariate boxplots, the most informative predictors for target (liver disease) are:
  - Bilirubin (TB, DB)
  - Alkaline Phosphatase (Alkphos)
  - Aspartate Aminotransferase
  - Albumin and A/G ratio (inversely related to liver disease)

MULTIVARIATE ANALYSIS

In [20]:
```
# Visualizing relationship between two input columns
# Compute correlation (only numeric columns)
corr = df.corr(numeric_only=True)

# Plot heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap of ILPD Dataset")
plt.show()
```



Observations:

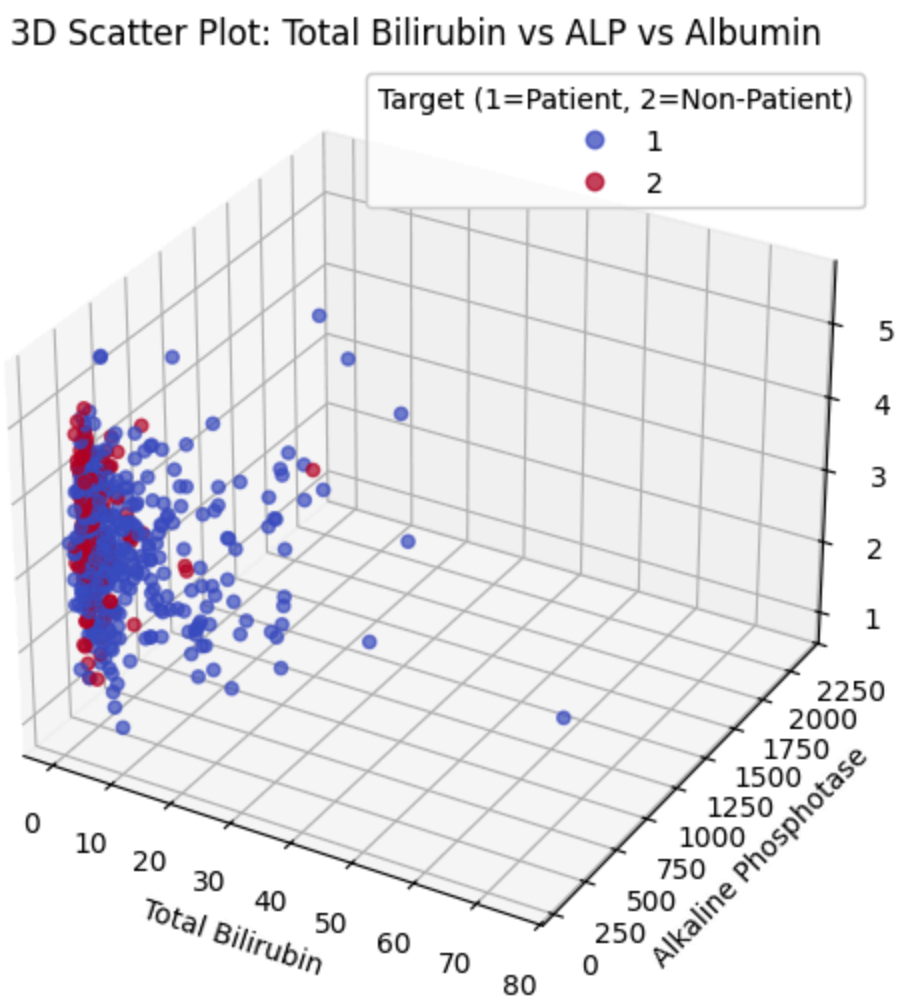
Based on the dataset properties:

- Total Bilirubin vs Direct Bilirubin: Strong positive correlation (close to 0.9). Makes sense because direct bilirubin is a component of total bilirubin. Direct bilirubin will be removed as it is highly correlated
- Albumin vs Albumin_and_Globulin_Ratio: Strong positive correlation (around 0.8). The ratio calculation involves albumin, so high dependency is expected.
- Liver Enzymes (Alkaline_Phosphotase, Alamine_Aminotransferase, Aspartate_Aminotransferase). These show moderate correlation among themselves. Indicates liver enzyme activity is somewhat related but not perfectly.
- Total Proteins vs Albumin: Moderate positive correlation (~0.6). Albumin is a part of total proteins, hence the relationship.
- Age vs Biochemical markers: Very weak or negligible correlations. Suggests liver disease markers are not strongly age-dependent in this dataset.

- Target Variable (Liver Disease: 1/2): No strong linear correlation with single features.

Implies we need multivariate models (logistic regression, random forest, etc.) for prediction rather than relying on one variable.

In [21]:
```
# Scatter Plot Using Three Variables
# Create 3D scatter plot
fig = plt.figure(figsize=(9,6))
ax = fig.add_subplot(111, projection='3d')

scatter = ax.scatter(
    df['Total_Bilirubin'],
    df['Alkaline_Phosphotase'],
    df['Albumin'],
    c=df['Target'], cmap='coolwarm', alpha=0.7
)

ax.set_xlabel("Total Bilirubin")
ax.set_ylabel("Alkaline Phosphotase")
ax.set_zlabel("Albumin")
plt.title("3D Scatter Plot: Total Bilirubin vs ALP vs Albumin")

# Add legend for Target
legend1 = ax.legend(*scatter.legend_elements(), title="Target (1=Patient, 2=Non-Pat
ax.add_artist(legend1)

plt.show()
```



Observations:

Insights from the above Scatter Plot:

- High Bilirubin + High ALP + Low Albumin → Liver disease
- Lower Bilirubin + Lower ALP + Higher Albumin → Healthy (non-patient)

Data Cleansing and Transformation

Handling null values, duplicates and categorical datatypes

In [22]:
```
# Computing and replacing missing /null values
df.fillna(value={'Albumin_and_Globulin_Ratio':df.Albumin_and_Globulin_Ratio .mean()})

# Rechecking for null values
df.isnull().sum()
```

Out[22]:
```
Age                          0
Gender                       0
Total_Bilirubin              0
Direct_Bilirubin             0
Alkaline_Phosphotase         0
Alamine_Aminotransferase     0
Aspartate_Aminotransferase   0
Total_Protiens               0
Albumin                      0
Albumin_and_Globulin_Ratio   0
Target                       0
dtype: int64
```

In [23]:
```
# Removing duplicated records
df.drop_duplicates(inplace=True)

# Rechecking for duplicated records
df.duplicated().sum()
```

Out[23]:
```
np.int64(0)
```

In [24]:
```
# Checking data shape after replacing null values and removing duplicates
df.shape
```

Out[24]:
```
(570, 11)
```

In [ ]:
```
# Encoding gender column into numerical
df['Gender']=df['Gender'].replace({'Male':1,'Female':0})

# checking first three rows to verify changes
df.head(3)
```

Out[ ]:

	Age	Gender	Total_Bilirubin	Direct_Bilirubin	Alkaline_Phosphotase	Alamine_Aminotrans
0	65	0	0.7	0.1	187	
1	62	1	10.9	5.5	699	
2	62	1	7.3	4.1	490	

In [ ]:
```
# Encoding the target columns to conventional way of 0 and 1:
# 0 rep negative (no liver disease) and 1 rep positive (liver disease)
df['Target']=df['Target'].replace({2:0,1:1})

# checking the value counts to verify transformation done correctly
df['Target'].value_counts()
```

Out[ ]:
```
Target
1    406
0    164
Name: count, dtype: int64
```

Splitting the Dataset and further transformation

In [27]:
```
# Library for dividing the data into training and testing sets
from sklearn.model_selection import train_test_split, train_test_split

# Separating features and target
y = df['Target']
x = df.drop(columns=['Target'])

# Split the dataset into training and testing sets
                    # --> test_size=0.2 means 20% test, 80% train
                    # --> stratify=y ensures class distribution in target is mainta
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42, stratify=y
)
```

Feature Selection

In [28]:
```
# Dropping columns with high multiconearity

# dropping columns in x_train set
x_train.drop(
    columns=[
        'Direct_Bilirubin',
        'Aspartate_Aminotransferase',
        'Total_Protiens',
        'Albumin'],
    axis=1, inplace=True
)

# dropping columns in x_test set
x_test.drop(
    columns=[
        'Direct_Bilirubin',
        'Aspartate_Aminotransferase',
        'Total_Protiens',
        'Albumin'],
    axis=1, inplace=True
)
```

Log Transformation

In [29]:
```
# Transforming data using logarithm the scaler to the x_train dataset and transform
#x_train['Age'] = np.log(x_train[['Age']])
x_train['Total_Bilirubin'] = np.log(x_train[['Total_Bilirubin']])
#x_train['Direct_Bilirubin'] = np.log(x_train[['Direct_Bilirubin']])
x_train['Alkaline_Phosphotase'] = np.log(x_train[['Alkaline_Phosphotase']])
x_train['Alamine_Aminotransferase'] = np.log(x_train[['Alamine_Aminotransferase']])
#x_train['Aspartate_Aminotransferase'] = np.log(x_train[['Aspartate_Aminotransferas
#x_train['Total_Protiens'] = np.log(x_train[['Total_Protiens']])
#x_train['Albumin'] = np.log(x_train[['Albumin']])
x_train['Albumin_and_Globulin_Ratio'] = np.log(x_train[['Albumin_and_Globulin_Ratio
```

Feature Scaling

In [30]:
```
# Library for feature scaling
from sklearn.preprocessing import RobustScaler

# Initializeing RobustScaler object
rs = RobustScaler()

# Renaming training and test sets to reflect scaling
x_train, x_test, y_train, y_test = x_train, x_test, y_train, y_test

# Fit the scaler to the x_train dataset and transform it
#x_train['Age'] = ss.fit_transform(x_train[['Age']])
x_train['Total_Bilirubin'] = rs.fit_transform(x_train[['Total_Bilirubin']])
#x_train['Direct_Bilirubin'] = rs.fit_transform(x_train[['Direct_Bilirubin']])
x_train['Alkaline_Phosphotase'] = rs.fit_transform(x_train[['Alkaline_Phosphotase']
x_train['Alamine_Aminotransferase'] = rs.fit_transform(x_train[['Alamine_Aminotrans
#x_train['Aspartate_Aminotransferase'] = rs.fit_transform(x_train[['Aspartate_Amino
#x_train['Total_Protiens'] = rs.fit_transform(x_train[['Total_Protiens']])
#x_train['Albumin'] = rs.fit_transform(x_train[['Albumin']])
x_train['Albumin_and_Globulin_Ratio'] = rs.fit_transform(x_train[['Albumin_and_Glob
```

Handling imbalance target column

In [31]:
```
# Check class distribution before SMOTENC:
print("Class Distribution before SMOTENC:")
print(y_train.value_counts())

# Importing library for balancing target column  --> SMOTENC technique is used
from imblearn.over_sampling import SMOTENC

# Initializing SMOTE object
smote = SMOTENC(categorical_features=['Gender'],random_state=42)

# Fitting SMOTE to oversample the minority class from training set
x_train, y_train = smote.fit_resample(x_train, y_train)

# Check class distribution after smotenc
print("\nClass Distrribution after SMOTENC:")
print(pd.Series(y_train).value_counts())
```

```
Class Distribution before SMOTENC:
Target
1    325
0    131
Name: count, dtype: int64

Class Distrribution after SMOTENC:
Target
1    325
0    325
Name: count, dtype: int64
```

MODEL BUILDING

Model Training & Evaluation

The models adopted by the team are described below:

- **1. Logistic Regression Models:** Commonly used to model linear relationship between inputs and a categorical output (1 or 0). Easy to implement, interprete, efficient to train and less prone to overfitting especially when regularization is used.
- **2. Decision Tree Models:** make decision rules on the features to produce predictions. They are frequently used for classification tasks and common in healthcare analytics. They excel at segmenting patient populations based on critical attributes.
- **3. Random Forest Models:** split data based on specific features and enhance this by combining multiple trees. Less prone to overfitting and touted for higher accuracy compared to other models
- **4. XGBoost Models:** efficient, flexible and can be used for classification tasks. captures non linear relationships in the datasets and also touted for high accuracy.
- **5. K-Nearest Neighbour (KNN) Models:** Work by calculating 'k' closes neighbors to a given data input and predict based majority class. They are non-parametric hence don't make assumption on underlying distribution of data. Commonly used in health analytics.
- **6. SVM Models:** Work by calculating optimal decision boundaries that maximize margins between different data classes. Touted for effectiveness in high-dimensional data and also handle non-linear data.

In [32]:
```
# Importing libraries for training models
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from xgboost import XGBClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

# Importing libraries for assessing performance accuracy of the models
from sklearn.metrics import accuracy_score,f1_score, recall_score, precision_score
from sklearn.metrics import classification_report,roc_curve, auc, roc_auc_score, co

# importing libraries for cross-validation
from sklearn.model_selection import cross_val_score
```

Training Without Using Hyperparameter Tuning

1. Logistic Regression

In [33]:
```
# Model Training
# Initialising logistic regression model
lr_model = LogisticRegression()

# Fitting model into training set
lr_model.fit(x_train, y_train)
```

Out[33]:

LogisticRegression

▶ Parameters

In [34]:
```
# Model Evaluation
# Predict y_train results
y_train_pred_lr=lr_model.predict(x_train)

# Predict test results
y_pred_lr=lr_model.predict(x_test)

# Generate ROC curve from y_test and pred
fpr, tpr, _ = roc_curve(y_test, y_pred_lr)

print(lr_model)
# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    CHECKING OVERFITTING    ============')
print("y_train prediction score: ", accuracy_score(y_train, y_train_pred_lr))
print('')
print('')

# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    PREDICTION SCORES FOR DIFFERENT METRICS    ============')
print("Accuracy score: \t", accuracy_score(y_test, y_pred_lr))
print("Precision score: \t", precision_score(y_test, y_pred_lr, average='weighted')
print("Recall score: \t\t",recall_score(y_test, y_pred_lr, average='weighted'))
print("F1 score: \t\t", f1_score(y_test, y_pred_lr, average='weighted'))
print("ROC AUC Score:\t\t", roc_auc_score(y_test, y_pred_lr))
print("AUC from roc_curve: \t", auc(fpr, tpr))
print('')
print('')

# Print a classification report comparing true labels (y_test) with predicted label
print('============    CLASSIFICATION REPORT    ============')
print(classification_report(y_test,y_pred_lr))
print('')
print('')

# Print a confusion matrix for the model
print('============    CONFUSION MATRIX    ============')
print(confusion_matrix(y_test, y_pred_lr))
```

```
LogisticRegression()
============    CHECKING OVERFITTING    ============
y_train prediction score:  0.7107692307692308


============    PREDICTION SCORES FOR DIFFERENT METRICS    ============
Accuracy score:          0.7105263157894737
Precision score:         0.5048476454293629
Recall score:            0.7105263157894737
F1 score:                0.5902834008097166
ROC AUC Score:           0.5
AUC from roc_curve:      0.5


============    CLASSIFICATION REPORT    ============
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        33
           1       0.71      1.00      0.83        81

    accuracy                           0.71       114
   macro avg       0.36      0.50      0.42       114
weighted avg       0.50      0.71      0.59       114



============    CONFUSION MATRIX    ============
[[ 0 33]
 [ 0 81]]
```

2. Decision Tree

In [35]:
```
# model training
# Initialising Decision Tree model
dt_model = DecisionTreeClassifier()

# Fitting model into training set
dt_model.fit(x_train, y_train)
```

Out[35]:
```
▾ DecisionTreeClassifier
▶ Parameters
```

In [36]:
```
# Model Evaluation
# Predict y_train results
y_train_pred_dt=dt_model.predict(x_train)

# Predict test results
y_pred_dt=dt_model.predict(x_test)

# Generate ROC curve from y_test and pred
fpr, tpr, _ = roc_curve(y_test, y_pred_dt)

print(dt_model)
```

```
# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    CHECKING OVERFITTING    ============')
print("y_train prediction score: ", accuracy_score(y_train, y_train_pred_dt))
print('')
print('')
# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    PREDICTION SCORES FOR DIFFERENT METRICS    ============')
print("Accuracy score: \t", accuracy_score(y_test, y_pred_dt))
print("Precision score: \t", precision_score(y_test, y_pred_dt, average='weighted')
print("Recall score: \t\t",recall_score(y_test, y_pred_dt, average='weighted'))
print("F1 score: \t\t", f1_score(y_test, y_pred_dt, average='weighted'))
print("ROC AUC Score:\t\t", roc_auc_score(y_test, y_pred_dt))
print("AUC from roc_curve: \t", auc(fpr, tpr))
print('')
print('')

# Print a classification report comparing true labels (y_test) with predicted label
print('============    CLASSIFICATION REPORT    ============')
print(classification_report(y_test,y_pred_dt))
print('')
print('')

# Print a confusion matrix for the model
print('============    CONFUSION MATRIX    ============')
print(confusion_matrix(y_test, y_pred_dt))
```

```
DecisionTreeClassifier()
============    CHECKING OVERFITTING    ============
y_train prediction score:  1.0


============    PREDICTION SCORES FOR DIFFERENT METRICS    ============
Accuracy score:          0.34210526315789475
Precision score:         0.44294436463366854
Recall score:            0.34210526315789475
F1 score:                0.370795409576573
ROC AUC Score:           0.33052749719416386
AUC from roc_curve:      0.33052749719416386


============    CLASSIFICATION REPORT    ============
              precision    recall  f1-score   support

           0       0.16      0.30      0.21        33
           1       0.56      0.36      0.44        81

    accuracy                           0.34       114
   macro avg       0.36      0.33      0.32       114
weighted avg       0.44      0.34      0.37       114



============    CONFUSION MATRIX    ============
[[10 23]
 [52 29]]
```

3. Random Forest Model

In [37]:
```
# Model Training
# Initializing Random Forest model
rf_model = RandomForestClassifier()

# Fitting model into training set
rf_model.fit(x_train, y_train)
```

Out[37]:
```
▾ RandomForestClassifier
▶ Parameters
```

In [38]:
```
# Model Evaluation
# Predict y_train results
y_train_pred_rf=rf_model.predict(x_train)

# Predict test results
y_pred_rf=rf_model.predict(x_test)

# Generate ROC curve from y_test and pred
fpr, tpr, _ = roc_curve(y_test, y_pred_rf)

print(rf_model)
# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    CHECKING OVERFITTING    ============')
print("y_train prediction score: ", accuracy_score(y_train, y_train_pred_rf))
print('')
print('')
# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    PREDICTION SCORES FOR DIFFERENT METRICS    ============')
print("Accuracy score: \t", accuracy_score(y_test, y_pred_rf))
print("Precision score: \t", precision_score(y_test, y_pred_rf, average='weighted'))
print("Recall score: \t\t",recall_score(y_test, y_pred_rf, average='weighted'))
print("F1 score: \t\t", f1_score(y_test, y_pred_rf, average='weighted'))
print("ROC AUC Score:\t\t", roc_auc_score(y_test, y_pred_rf))
print("AUC from roc_curve: \t", auc(fpr, tpr))
print('')
print('')

# Print a classification report comparing true labels (y_test) with predicted label
print('============    CLASSIFICATION REPORT    ============')
print(classification_report(y_test,y_pred_rf))
print('')
print('')

# Print a confusion matrix for the model
print('============    CONFUSION MATRIX    ============')
print(confusion_matrix(y_test, y_pred_rf))
```

```
RandomForestClassifier()
============    CHECKING OVERFITTING    ============
y_train prediction score:  1.0


============    PREDICTION SCORES FOR DIFFERENT METRICS    ============
Accuracy score:          0.7105263157894737
Precision score:         0.5048476454293629
Recall score:            0.7105263157894737
F1 score:                0.5902834008097166
ROC AUC Score:           0.5
AUC from roc_curve:      0.5


============    CLASSIFICATION REPORT    ============
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        33
           1       0.71      1.00      0.83        81

    accuracy                           0.71       114
   macro avg       0.36      0.50      0.42       114
weighted avg       0.50      0.71      0.59       114



============    CONFUSION MATRIX    ============
[[ 0 33]
 [ 0 81]]
```

4. XGBOOST

In [39]:
```
# Model Training
# Inialiazing XGBoost Model
xgb_model = XGBClassifier()

# Fitting model into training set
xgb_model.fit(x_train, y_train)
```

Out[39]:

▾ XGBClassifier

▶ Parameters

In [40]:
```
# Model Evaluation
# Predict y_train results
y_train_pred=xgb_model.predict(x_train)

# Predict test results
y_pred=xgb_model.predict(x_test)

# Generate ROC curve from y_test and pred
fpr, tpr, _ = roc_curve(y_test, y_pred)

print(rf_model)
```

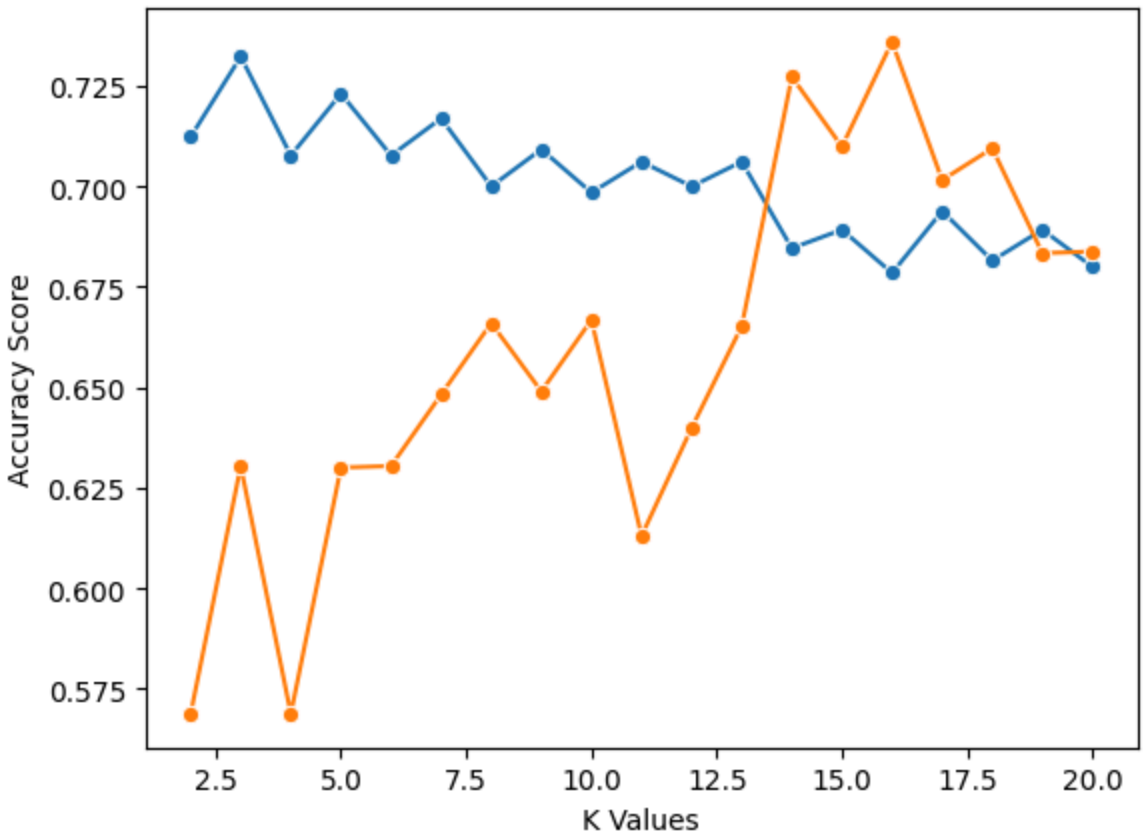
```
# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    CHECKING OVERFITTING    ============')
print("y_train prediction score: ", accuracy_score(y_train, y_train_pred))
print('')
print('')
# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    PREDICTION SCORES FOR DIFFERENT METRICS    ============')
print("Accuracy score: \t", accuracy_score(y_test, y_pred))
print("Precision score: \t", precision_score(y_test, y_pred, average='weighted'))
print("Recall score: \t\t",recall_score(y_test, y_pred, average='weighted'))
print("F1 score: \t\t", f1_score(y_test, y_pred, average='weighted'))
print("ROC AUC Score:\t\t", roc_auc_score(y_test, y_pred))
print("AUC from roc_curve: \t", auc(fpr, tpr))
print('')
print('')

# Print a classification report comparing true labels (y_test) with predicted label
print('============    CLASSIFICATION REPORT    ============')
print(classification_report(y_test,y_pred))
print('')
print('')

# Print a confusion matrix for the model
print('============    CONFUSION MATRIX    ============')
print(confusion_matrix(y_test, y_pred))
```

```
RandomForestClassifier()
============    CHECKING OVERFITTING    ============
y_train prediction score:  1.0


============    PREDICTION SCORES FOR DIFFERENT METRICS    ============
Accuracy score:          0.7105263157894737
Precision score:         0.5048476454293629
Recall score:            0.7105263157894737
F1 score:                0.5902834008097166
ROC AUC Score:           0.5
AUC from roc_curve:      0.5


============    CLASSIFICATION REPORT    ============
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        33
           1       0.71      1.00      0.83        81

    accuracy                           0.71       114
   macro avg       0.36      0.50      0.42       114
weighted avg       0.50      0.71      0.59       114



============    CONFUSION MATRIX    ============
[[ 0 33]
 [ 0 81]]
```

5. KNN

In [41]:
```
# Finding optimal value of K

# create list variables to store scores for each value of k
k_values = [i for i in range(2,21)]
scores_trn = []
scores_tst = []

# loop the different values of K
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)

    train_score = cross_val_score(knn, x_train, y_train, cv=5)
    scores_trn.append(np.mean(train_score))

    test_score = cross_val_score(knn, x_test, y_test, cv=5)
    scores_tst.append(np.mean(test_score))


# plotting
sns.lineplot(x = k_values, y = scores_trn, marker = 'o')
sns.lineplot(x = k_values, y = scores_tst, marker = 'o')
plt.xlabel("K Values")
plt.ylabel("Accuracy Score")
```

Out[41]:
```
Text(0, 0.5, 'Accuracy Score')
```



In [42]:
```
# Model Training
# Initializing KNN model with optimal value of K
knn_model = KNeighborsClassifier(n_neighbors=15)

# Fitting model into training set
knn_model.fit(x_train, y_train)
```

Out[42]:

KNeighborsClassifier

▶ Parameters

In [43]:
```
# Model Evaluation
# Predict y_train results
y_train_pred=knn_model.predict(x_train)

# Predict test results
y_pred=knn_model.predict(x_test)

# Generate ROC curve from y_test and pred
fpr, tpr, _ = roc_curve(y_test, y_pred)

print(rf_model)
# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    CHECKING OVERFITTING    ============')
print("y_train prediction score: ", accuracy_score(y_train, y_train_pred))
print('')
print('')
# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    PREDICTION SCORES FOR DIFFERENT METRICS    ============')
print("Accuracy score: \t", accuracy_score(y_test, y_pred))
print("Precision score: \t", precision_score(y_test, y_pred, average='weighted'))
print("Recall score: \t\t",recall_score(y_test, y_pred, average='weighted'))
print("F1 score: \t\t", f1_score(y_test, y_pred, average='weighted'))
print("ROC AUC Score:\t\t", roc_auc_score(y_test, y_pred))
print("AUC from roc_curve: \t", auc(fpr, tpr))
print('')
print('')

# Print a classification report comparing true labels (y_test) with predicted label
print('============    CLASSIFICATION REPORT    ============')
print(classification_report(y_test,y_pred))
print('')
print('')

# Print a confusion matrix for the model
print('============    CONFUSION MATRIX    ============')
print(confusion_matrix(y_test, y_pred))
```

```
RandomForestClassifier()
============    CHECKING OVERFITTING    ============
y_train prediction score:  0.74


============    PREDICTION SCORES FOR DIFFERENT METRICS    ============
Accuracy score:          0.7105263157894737
Precision score:         0.5048476454293629
Recall score:            0.7105263157894737
F1 score:                0.5902834008097166
ROC AUC Score:           0.5
AUC from roc_curve:      0.5


============    CLASSIFICATION REPORT    ============
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        33
           1       0.71      1.00      0.83        81

    accuracy                           0.71       114
   macro avg       0.36      0.50      0.42       114
weighted avg       0.50      0.71      0.59       114



============    CONFUSION MATRIX    ============
[[ 0 33]
 [ 0 81]]
```

6. SVM

In [44]:
```
# Model Training
# Initializing SVM Model
svm_model = SVC()

# Fitting model into training set
svm_model.fit(x_train, y_train)
```

Out[44]:

▾ SVC

▶ Parameters

In [45]:
```
# Model Evaluation
# Predict y_train results
y_train_pred=svm_model.predict(x_train)

# Predict test results
y_pred=svm_model.predict(x_test)

# Generate ROC curve from y_test and pred
fpr, tpr, _ = roc_curve(y_test, y_pred)

print(rf_model)
```

```
# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    CHECKING OVERFITTING    ============')
print("y_train prediction score: ", accuracy_score(y_train, y_train_pred))
print('')
print('')

# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    PREDICTION SCORES FOR DIFFERENT METRICS    ============')
print("Accuracy score: \t", accuracy_score(y_test, y_pred))
print("Precision score: \t", precision_score(y_test, y_pred, average='weighted'))
print("Recall score: \t\t",recall_score(y_test, y_pred, average='weighted'))
print("F1 score: \t\t", f1_score(y_test, y_pred, average='weighted'))
print("ROC AUC Score:\t\t", roc_auc_score(y_test, y_pred))
print("AUC from roc_curve: \t", auc(fpr, tpr))
print('')
print('')

# Print a classification report comparing true labels (y_test) with predicted label
print('============    CLASSIFICATION REPORT    ============')
print(classification_report(y_test,y_pred))
print('')
print('')

# Print a confusion matrix for the model
print('============    CONFUSION MATRIX    ============')
print(confusion_matrix(y_test, y_pred))
```

```
RandomForestClassifier()
============    CHECKING OVERFITTING    ============
y_train prediction score:  0.6230769230769231


============    PREDICTION SCORES FOR DIFFERENT METRICS    ============
Accuracy score:          0.7105263157894737
Precision score:         0.5048476454293629
Recall score:            0.7105263157894737
F1 score:                0.5902834008097166
ROC AUC Score:           0.5
AUC from roc_curve:      0.5


============    CLASSIFICATION REPORT    ============
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        33
           1       0.71      1.00      0.83        81

    accuracy                           0.71       114
   macro avg       0.36      0.50      0.42       114
weighted avg       0.50      0.71      0.59       114



============    CONFUSION MATRIX    ============
[[ 0 33]
 [ 0 81]]
```

Training Using Hyperparameter Tuning

In [46]:
```
# Importing Hyperparameter tuning library  --> GridSearchCV from sklearn
from sklearn.model_selection import GridSearchCV
```

1. Optimized Logistic Regression

In [47]:
```
# Hyperparameter Tuning
# Defining the hyperparameter grid
param_grid = [{
    'penalty':['l1','l2','elasticnet','none'],                  # regularization al
    'C' : np.logspace(-3,3,20),                                 # regularization st
    'solver': ['lbfgs','newton-cg','liblinear','sag','saga'],   # algorithm that mi
    'max_iter'  : [100,1000,2500,5000]                          # maximum number of
}]

# Initializing GridSearchCV object
grid = GridSearchCV(LogisticRegression(),param_grid = param_grid, refit = True,scor

# Fitting the GridSearchCV object into training set
grid.fit(x_train,y_train)

# Print the best set of hyperparameters and score from grid search
print("Best Parameters:", grid.best_params_)
print("Best Estimator:", grid.best_estimator_)
print("Best score: ", grid.best_score_)
```

```
Fitting 5 folds for each of 1600 candidates, totalling 8000 fits
Best Parameters: {'C': np.float64(0.004281332398719396), 'max_iter': 100, 'penalty':
'l2', 'solver': 'saga'}
Best Estimator: LogisticRegression(C=np.float64(0.004281332398719396), solver='sag
a')
Best score:  0.689390864123121
```

In [48]:
```
# Model Training
# Initializing model with best parameters
lr_opt_model = LogisticRegression(C= grid.best_params_['C'],
                                  max_iter= grid.best_params_['max_iter'],
                                  penalty= grid.best_params_['penalty'],
                                  solver= grid.best_params_['solver'])

# Fitting model with the best hyperparameters into training set
lr_opt_model.fit(x_train,y_train)
```

Out[48]:

▾ LogisticRegression

▶ Parameters

In [49]:
```
grid.best_params_['penalty']
```

Out[49]: 'l2'

In [50]:
```
# Model Evaluation
# Predict y_train results using best model from grid search
y_train_pred=lr_opt_model.predict(x_train)

# Predict test results using best model from grid search
y_pred=lr_opt_model.predict(x_test)

# Generate ROC curve from y_test and pred
fpr, tpr, _ = roc_curve(y_test, y_pred)

# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    CHECKING OVERFITTING    ============')
print("y_train prediction score: ", accuracy_score(y_train, y_train_pred))
print('')
print('')

# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    PREDICTION SCORES FOR DIFFERENT METRICS    ============')
print("Accuracy score: \t", accuracy_score(y_test, y_pred))
print("Precision score: \t", precision_score(y_test, y_pred, average='weighted'))
print("Recall score: \t\t",recall_score(y_test, y_pred, average='weighted'))
print("F1 score: \t\t", f1_score(y_test, y_pred, average='weighted'))
print("ROC AUC Score:\t\t", roc_auc_score(y_test, y_pred))
print("AUC from roc_curve: \t", auc(fpr, tpr))
print('')
print('')

# Print a classification report comparing true labels (y_test) with predicted label
print('============    CLASSIFICATION REPORT    ============')
print(classification_report(y_test,y_pred))
print('')
print('')

# Print a confusion matrix for the model
print('============    CONFUSION MATRIX    ============')
print(confusion_matrix(y_test, y_pred))
```

```
============    CHECKING OVERFITTING    ============
y_train prediction score:  0.7230769230769231


============    PREDICTION SCORES FOR DIFFERENT METRICS    ============
Accuracy score:          0.7105263157894737
Precision score:         0.5048476454293629
Recall score:            0.7105263157894737
F1 score:                0.5902834008097166
ROC AUC Score:           0.5
AUC from roc_curve:      0.5


============    CLASSIFICATION REPORT    ============
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        33
           1       0.71      1.00      0.83        81

    accuracy                           0.71       114
   macro avg       0.36      0.50      0.42       114
weighted avg       0.50      0.71      0.59       114



============    CONFUSION MATRIX    ============
[[ 0 33]
 [ 0 81]]
```

2. Optimized Decision Tree

In [51]:
```
# Hyperparameter Tuning
# Defining the hyperparameter grid
param_grid = {
    "criterion":("gini", "entropy"),        # quality of split
    "splitter":("best", "random"),          # searches the features for a split
    "max_depth":(list(range(1, 10))),       # depth of tree range from 1 to 9
    "min_samples_split":[2, 3, 4,5,6,7],    # the minimum number of samples require
    "min_samples_leaf":list(range(1, 10)),  # minimum number of samples required to
}

# Initializing GridSearchCV object
grid = GridSearchCV(DecisionTreeClassifier(), param_grid = param_grid, refit = True

# Fitting the GridSearchCV object into training set
grid.fit(x_train,y_train)

# Print the best set of hyperparameters and score
print("Best Parameters:", grid.best_params_)
print("Best Estimator:", grid.best_estimator_)
print("Best score: ", grid.best_score_)
```

```
Fitting 5 folds for each of 1944 candidates, totalling 9720 fits
Best Parameters: {'criterion': 'entropy', 'max_depth': 8, 'min_samples_leaf': 7, 'mi
n_samples_split': 2, 'splitter': 'random'}
Best Estimator: DecisionTreeClassifier(criterion='entropy', max_depth=8, min_samples
_leaf=7,
                       splitter='random')
Best score:  0.7256373478403805
```

In [52]:
```
# Model Training with the best hyperparameters
# initialising model with best parameters
dt_opt_model = DecisionTreeClassifier(criterion=grid.best_params_['criterion'],
                                      max_depth=grid.best_params_['max_depth'],
                                      min_samples_leaf= grid.best_params_['min_samp
                                      min_samples_split=grid.best_params_['min_samp
                                      splitter=grid.best_params_['splitter'])

# Fitting model with best hyperparameter into training set
dt_opt_model.fit(x_train,y_train)
```

Out[52]:

▾ DecisionTreeClassifier

▶ Parameters

In [53]:
```
# Model Evaluation
# Predict y_train results using best model from grid search
y_train_pred=dt_opt_model.predict(x_train)

# Predict test results using best model from grid search
y_pred=dt_opt_model.predict(x_test)

# Generate ROC curve from y_test and pred
fpr, tpr, _ = roc_curve(y_test, y_pred)

# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    CHECKING OVERFITTING    ============')
print("y_train prediction score: ", accuracy_score(y_train, y_train_pred))
print('')
print('')
# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    PREDICTION SCORES FOR DIFFERENT METRICS    ============')
print("Accuracy score: \t", accuracy_score(y_test, y_pred))
print("Precision score: \t", precision_score(y_test, y_pred, average='weighted'))
print("Recall score: \t\t",recall_score(y_test, y_pred, average='weighted'))
print("F1 score: \t\t", f1_score(y_test, y_pred, average='weighted'))
print("ROC AUC Score:\t\t", roc_auc_score(y_test, y_pred))
print("AUC from roc_curve: \t", auc(fpr, tpr))
print('')
print('')

# Print a classification report comparing true labels (y_test) with predicted label
print('============    CLASSIFICATION REPORT    ============')
print(classification_report(y_test,y_pred))
print('')
print('')
```

```
# Print a confusion matrix for the model
print('============    CONFUSION MATRIX    ============')
print(confusion_matrix(y_test, y_pred))
```

```
============    CHECKING OVERFITTING    ============
y_train prediction score:  0.7138461538461538


============    PREDICTION SCORES FOR DIFFERENT METRICS    ============
Accuracy score:          0.7105263157894737
Precision score:         0.5048476454293629
Recall score:            0.7105263157894737
F1 score:                0.5902834008097166
ROC AUC Score:           0.5
AUC from roc_curve:      0.5


============    CLASSIFICATION REPORT    ============
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        33
           1       0.71      1.00      0.83        81

    accuracy                           0.71       114
   macro avg       0.36      0.50      0.42       114
weighted avg       0.50      0.71      0.59       114



============    CONFUSION MATRIX    ============
[[ 0 33]
 [ 0 81]]
```

3. Optimized Random Forest

In [54]:
```
# Hyperparameter Tuning
# Defining the hyperparameter grid
param_grid = {
    'n_estimators': [50,100,200,300,400,500],       # Total number of trees, more t
    'max_depth': [2,3, 5, 10, 15],                  # maximum levels in decision tr
    'min_samples_split': [2, 5,10],                 # minimum number of samples req
    'min_samples_leaf': [1, 2,4]                    # minimum number of samples req
 }

# Initializing GridSearchCV object
grid = GridSearchCV(RandomForestClassifier(),param_grid = param_grid, refit = True,

# Fitting the GridSearchCV object into the training set
grid.fit(x_train,y_train)

# Print the best set of hyperparameters and score
print("Best Parameters:", grid.best_params_)
print("Best Estimator:", grid.best_estimator_)
print("Best score: ", grid.best_score_)
```

```
Fitting 5 folds for each of 270 candidates, totalling 1350 fits
Best Parameters: {'max_depth': 15, 'min_samples_leaf': 1, 'min_samples_split': 5, 'n
_estimators': 100}
Best Estimator: RandomForestClassifier(max_depth=15, min_samples_split=5)
Best score:  0.7769924192144773
```

In [55]:
```
# Model Training with the best hyperparameters
# initialising model with best parameters
rf_opt_model = RandomForestClassifier(max_depth= grid.best_params_['max_depth'],
                                      min_samples_leaf= grid.best_params_['min_samp
                                      min_samples_split= grid.best_params_['min_sam
                                      n_estimators= grid.best_params_['n_estimators

# Fitting model with best paramas into training set
rf_opt_model.fit(x_train,y_train)
```

Out[55]:

RandomForestClassifier

▶ Parameters

In [56]:
```
# Model Evaluation
# Predict y_train results using best model from grid search
y_train_pred=rf_opt_model.predict(x_train)

# Predict test results using best model from grid search
y_pred=rf_opt_model.predict(x_test)

# Generate ROC curve from y_test and pred
fpr, tpr, _ = roc_curve(y_test, y_pred)

# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    CHECKING OVERFITTING    ============')
print("y_train prediction score: ", accuracy_score(y_train, y_train_pred))
print('')
print('')
# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    PREDICTION SCORES FOR DIFFERENT METRICS    ============')
print("Accuracy score: \t", accuracy_score(y_test, y_pred))
print("Precision score: \t", precision_score(y_test, y_pred, average='weighted'))
print("Recall score: \t\t",recall_score(y_test, y_pred, average='weighted'))
print("F1 score: \t\t", f1_score(y_test, y_pred, average='weighted'))
print("ROC AUC Score:\t\t", roc_auc_score(y_test, y_pred))
print("AUC from roc_curve: \t", auc(fpr, tpr))
print('')
print('')

# Print a classification report comparing true labels (y_test) with predicted label
print('============    CLASSIFICATION REPORT    ============')
print(classification_report(y_test,y_pred))
print('')
print('')

# Print a confusion matrix for the model
```

```
print('============    CONFUSION MATRIX    ============')
print(confusion_matrix(y_test, y_pred))
```

```
============    CHECKING OVERFITTING    ============
y_train prediction score:  0.9938461538461538


============    PREDICTION SCORES FOR DIFFERENT METRICS    ============
Accuracy score:          0.7105263157894737
Precision score:         0.5048476454293629
Recall score:            0.7105263157894737
F1 score:                0.5902834008097166
ROC AUC Score:           0.5
AUC from roc_curve:      0.5


============    CLASSIFICATION REPORT    ============
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        33
           1       0.71      1.00      0.83        81

    accuracy                           0.71       114
   macro avg       0.36      0.50      0.42       114
weighted avg       0.50      0.71      0.59       114



============    CONFUSION MATRIX    ============
[[ 0 33]
 [ 0 81]]
```

4. Optimized XGBoost

In [57]:
```
# Hyperparameter Tuning
# Defining the hyperparameter grid
param_grid = {
    'n_estimators': [50,100,200,300,400,500],
    'max_depth': [2,3, 5, 10, 15],
    'learning_rate': [0.5,0.1, 0.01, 0.001],
    'reg_lambda': [0.001, 0.1, 1.0, 10.0, 100.0],
    'subsample': [0.5, 0.7,10]
}

# Initializing GridSearchCV object
grid = GridSearchCV(XGBClassifier(),param_grid = param_grid, refit = True, scoring=

# Fitting the GridSearchCV object into the training set
grid.fit(x_train,y_train)

# Print the best set of hyperparameters and score
print("Best Parameters:", grid.best_params_)
print("Best Estimator:", grid.best_estimator_)
print("Best score: ", grid.best_score_)
```

```
Fitting 5 folds for each of 1800 candidates, totalling 9000 fits
Best Parameters: {'learning_rate': 0.5, 'max_depth': 5, 'n_estimators': 200, 'reg_la
mbda': 0.001, 'subsample': 0.7}
Best Estimator: XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None, feature_types=None,
              feature_weights=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=0.5, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None, max_depth=5,
              max_leaves=None, min_child_weight=None, missing=nan,
              monotone_constraints=None, multi_strategy=None, n_estimators=200,
              n_jobs=None, num_parallel_tree=None, ...)
Best score:  0.8010104072497961
```

In [58]:
```
# Building model with the best hyperparameters
# initialising model with best parameters
xgb_opt_model = XGBClassifier(learning_rate= grid.best_params_['learning_rate'],
                              max_depth= grid.best_params_['max_depth'],
                              n_estimators=grid.best_params_['n_estimators'],
                              reg_lambda=grid.best_params_['reg_lambda'],
                              subsample= grid.best_params_['subsample'])

# training model with best parameter
xgb_opt_model.fit(x_train,y_train)
```

Out[58]:

▾ XGBClassifier

▶ Parameters

In [59]:
```
# Model EValuation
# Predict y_train results using best model from grid search
y_train_pred=xgb_opt_model.predict(x_train)

# Predict test results using best model from grid search
y_pred=xgb_opt_model.predict(x_test)

# Generate ROC curve from y_test and pred
fpr, tpr, _ = roc_curve(y_test, y_pred)

# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    CHECKING OVERFITTING    ============')
print("y_train prediction score: ", accuracy_score(y_train, y_train_pred))
print('')
print('')
# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    PREDICTION SCORES FOR DIFFERENT METRICS    ============')
print("Accuracy score: \t", accuracy_score(y_test, y_pred))
print("Precision score: \t", precision_score(y_test, y_pred, average='weighted'))
print("Recall score: \t\t",recall_score(y_test, y_pred, average='weighted'))
print("F1 score: \t\t", f1_score(y_test, y_pred, average='weighted'))
print("ROC AUC Score:\t\t", roc_auc_score(y_test, y_pred))
print("AUC from roc_curve: \t", auc(fpr, tpr))
print('')
```

```
print('')

# Print a classification report comparing true labels (y_test) with predicted label
print('============    CLASSIFICATION REPORT    ============')
print(classification_report(y_test,y_pred))
print('')
print('')

# Print a confusion matrix for the model
print('============    CONFUSION MATRIX    ============')
print(confusion_matrix(y_test, y_pred))
```

```
============    CHECKING OVERFITTING    ============
y_train prediction score:  1.0


============    PREDICTION SCORES FOR DIFFERENT METRICS    ============
Accuracy score:          0.7017543859649122
Precision score:         0.5030274802049371
Recall score:            0.7017543859649122
F1 score:                0.5860010851871947
ROC AUC Score:           0.49382716049382713
AUC from roc_curve:      0.49382716049382713


============    CLASSIFICATION REPORT    ============
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        33
           1       0.71      0.99      0.82        81

    accuracy                           0.70       114
   macro avg       0.35      0.49      0.41       114
weighted avg       0.50      0.70      0.59       114



============    CONFUSION MATRIX    ============
[[ 0 33]
 [ 1 80]]
```

5. Optimized KNN

In [60]:
```
# Hyperparameter Tuning
# Defining the hyperparameter grid
param_grid = {
    'n_neighbors': range(1,20),
    'weights': ['uniform', 'distance'],
    'metrics': ['euclidean', 'manhattan'],
    'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
    'p': [1, 2]
}

param_grid = {'n_neighbors': np.arange(1, 11),
              'weights': ['uniform', 'distance'],
              'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
```

```
              'p': [1, 2]}

# Initializing GridSearchCV object
grid = GridSearchCV(KNeighborsClassifier(),param_grid = param_grid, refit = True, s

# Fitting the GridSearchCV object to the training data
grid.fit(x_train,y_train)

# Print the best set of hyperparameters and score
print("Best Parameters:", grid.best_params_)
print("Best Estimator:", grid.best_estimator_)
print("Best score: ", grid.best_score_)
```

```
Fitting 5 folds for each of 160 candidates, totalling 800 fits
Best Parameters: {'algorithm': 'auto', 'n_neighbors': np.int64(1), 'p': 2, 'weight
s': 'uniform'}
Best Estimator: KNeighborsClassifier(n_neighbors=np.int64(1))
Best score:  0.7496573536734568
```

In [61]:
```
# Building model with the best hyperparameters
# initialising model with best parameters
knn_opt_model = KNeighborsClassifier(algorithm=grid.best_params_['algorithm'],
                                     n_neighbors=grid.best_params_['n_neighbors'],
                                     p=grid.best_params_['p'],
                                     weights=grid.best_params_['weights'])


# Fitting model with best parameter into training set
knn_opt_model.fit(x_train,y_train)
```

Out[61]:

KNeighborsClassifier

▶ Parameters

In [62]:
```
# Model Evaluation
# Predict y_train results using best model from grid search
y_train_pred=knn_opt_model.predict(x_train)

# Predict test results using best model from grid search
y_pred=knn_opt_model.predict(x_test)

# Generate ROC curve from y_test and pred
fpr, tpr, _ = roc_curve(y_test, y_pred)

# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    CHECKING OVERFITTING    ============')
print("y_train prediction score: ", accuracy_score(y_train, y_train_pred))
print('')
print('')
# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    PREDICTION SCORES FOR DIFFERENT METRICS    ============')
print("Accuracy score: \t", accuracy_score(y_test, y_pred))
print("Precision score: \t", precision_score(y_test, y_pred, average='weighted'))
print("Recall score: \t\t",recall_score(y_test, y_pred, average='weighted'))
print("F1 score: \t\t", f1_score(y_test, y_pred, average='weighted'))
```

```
print("ROC AUC Score:\t\t", roc_auc_score(y_test, y_pred))
print("AUC from roc_curve: \t", auc(fpr, tpr))
print('')
print('')

# Print a classification report comparing true labels (y_test) with predicted label
print('============    CLASSIFICATION REPORT    ============')
print(classification_report(y_test,y_pred))
print('')
print('')

# Print a confusion matrix for the model
print('============    CONFUSION MATRIX    ============')
print(confusion_matrix(y_test, y_pred))
```

```
============    CHECKING OVERFITTING    ============
y_train prediction score:  1.0


============    PREDICTION SCORES FOR DIFFERENT METRICS    ============
Accuracy score:          0.6929824561403509
Precision score:         0.5011748120300752
Recall score:            0.6929824561403509
F1 score:                0.5816743932369784
ROC AUC Score:           0.4876543209876543
AUC from roc_curve:      0.4876543209876543


============    CLASSIFICATION REPORT    ============
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        33
           1       0.71      0.98      0.82        81

    accuracy                           0.69       114
   macro avg       0.35      0.49      0.41       114
weighted avg       0.50      0.69      0.58       114



============    CONFUSION MATRIX    ============
[[ 0 33]
 [ 2 79]]
```

6. Optimized SVM

In [63]:
```
# Hyperparameter Tuning
# Defining the hyperparameter grid
param_grid = {
    'C': [0.1, 1, 10],
    'gamma': ['scale', 'auto', 0.1, 1],
    'degree': [2, 3, 4],
    'kernel': ['linear', 'rbf']
}
```

```
# Initializing GridSearchCV object
grid = GridSearchCV(SVC(),param_grid = param_grid, refit = True, scoring="f1", verb

# Fitting the GridSearchCV object to the training data
grid.fit(x_train,y_train)

# Print the best set of hyperparameters and score
print("Best Parameters:", grid.best_params_)
print("Best Estimator:", grid.best_estimator_)
print("Best score: ", grid.best_score_)
```

```
Fitting 5 folds for each of 72 candidates, totalling 360 fits
Best Parameters: {'C': 10, 'degree': 2, 'gamma': 1, 'kernel': 'rbf'}
Best Estimator: SVC(C=10, degree=2, gamma=1)
Best score:  0.809685155101725
```

In [64]:
```
# Model Training with the best hyperparameters
# initialising model with best parameters
svm_opt_model = SVC(C = grid.best_params_['C'],
                    gamma = grid.best_params_['gamma'],
                    kernel=grid.best_params_['kernel'],)

# Fitting model with best parameter into training set
svm_opt_model.fit(x_train,y_train)
```

Out[64]:

SVC

▶ Parameters

In [65]:
```
# Model Evaluation
# Predict y_train results using best model from grid search
y_train_pred=svm_opt_model.predict(x_train)

# Predict test results using best model from grid search
y_pred=svm_opt_model.predict(x_test)

# Generate ROC curve from y_test and pred
fpr, tpr, _ = roc_curve(y_test, y_pred)

# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    CHECKING OVERFITTING    ============')
print("y_train prediction score: ", accuracy_score(y_train, y_train_pred))
print('')
print('')
# Print model performance scores comparing true labels (y_test) with predicted labe
print('============    PREDICTION SCORES FOR DIFFERENT METRICS    ============')
print("Accuracy score: \t", accuracy_score(y_test, y_pred))
print("Precision score: \t", precision_score(y_test, y_pred, average='weighted'))
print("Recall score: \t\t",recall_score(y_test, y_pred, average='weighted'))
print("F1 score: \t\t", f1_score(y_test, y_pred, average='weighted'))
print("ROC AUC Score:\t\t", roc_auc_score(y_test, y_pred))
print("AUC from roc_curve: \t", auc(fpr, tpr))
print('')
print('')
```

```
# Print a classification report comparing true labels (y_test) with predicted label
print('============    CLASSIFICATION REPORT    ============')
print(classification_report(y_test,y_pred))
print('')
print('')

# Print a confusion matrix for the model
print('============    CONFUSION MATRIX    ============')
print(confusion_matrix(y_test, y_pred))
```

```
============    CHECKING OVERFITTING    ============
y_train prediction score:  0.9938461538461538


============    PREDICTION SCORES FOR DIFFERENT METRICS    ============
Accuracy score:          0.7105263157894737
Precision score:         0.5048476454293629
Recall score:            0.7105263157894737
F1 score:                0.5902834008097166
ROC AUC Score:           0.5
AUC from roc_curve:      0.5


============    CLASSIFICATION REPORT    ============
              precision    recall  f1-score   support

           0       0.00      0.00      0.00        33
           1       0.71      1.00      0.83        81

    accuracy                           0.71       114
   macro avg       0.36      0.50      0.42       114
weighted avg       0.50      0.71      0.59       114



============    CONFUSION MATRIX    ============
[[ 0 33]
 [ 0 81]]
```

Best Model for Production

In [ ]:
```
# Logistic regression with hyperparameters tuned is choosen as the best model for p


# Predict using the loaded model
sample_data = pd.DataFrame([{'Age': 54,
                             'Gender': 1,
                             'Total_Bilirubin': 10.5,
                             'Alkaline_Phosphotase': 671,
                             'Alamine_Aminotransferase': 46,
                             'Albumin_and_Globulin_Ratio': 0.85,
                             }])
prediction = lr_opt_model.predict(sample_data)
print(f"Prediction: {'Liver Disease Detected' if prediction[0] == 1 else 'Liver Dis
```

```
Prediction: Liver Disease Detected
```

ANALYSIS REPORT

In preparation of the final dataset for training and testing various models, the team performed the following transformation:

- replaced null values with mean of respective columns
- dropped duplicated rows
- encoded gender column into 0 representing female and 1 representing male
- encoded target column values into 0 and 1. This is in line with good practice where 1 represent presence of liver disease and 0 represent absence of liver disease
- dropped columns with high multicollinearity. This was aimed to help the model learn efficiently
- transformed continuous columns that exhibited high outliers using log transformation technique
- scaled continuous columns (x_train) that exhibited high variance using robust scaler
- applied SMOTENC (training set) to balance the target variable.

After the above tranformation, various models were trained and evaluated. The models were initially train without hyperparameter tuning and after trianed with best parameters from hyperparameter tuning. Here are performance results from the different algorithms:

Summary Performance Accuracy with NO hyperparameter tuning

Evaluation Metric	Logistic Regression	Decision Tree	Random Forest	XGB Classifier	K-Nearest Neighbours	Support Vector Machine
Accuracy score (training):	0.7108	1.0000	1.0000	1.0000	0.7400	0.6231
Accuracy score (test):	0.7105	0.3421	0.7105	0.7105	0.7105	0.7105
Precision score:	0.5048	0.4429	0.5048	0.5048	0.5048	0.5048
Recall score:	0.7105	0.3421	0.7105	0.7105	0.7105	0.7105
F1 score:	0.5903	0.3708	0.5903	0.5903	0.5903	0.5903
ROC AUC Score:	0.5000	0.3305	0.5000	0.5000	0.5000	0.5000
AUC from roc_curve:	0.5000	0.3305	0.5000	0.5000	0.5000	0.5000
confusion Matrix	[[0 33] [0 81]]	[[10 23] [52 29]]	[[0 33] [0 81]]	[[0 33] [0 81]]	[[0 33] [0 81]]	[[0 33] [0 81]]

Model training without hyperparameter tuning:

- Logistic regression and KNN performed relatively well both achieving 71% prediction of the test outcome and 71% and 74% prediction of y_train respectively.
- Decision Tree, Random Forest and XGBoost all exhibited overfitting. SVM performed badly in predicting y_train.
- The confusion matrix shows the distribution of predicted classes compared to the actual classes. Safe for Decision Trees the distribution in other algorithms was similar.

Summary Performance Accuracy with Hyperparameters Tuning

Evaluation Metric	Logistic Regression	Decision Tree	Random Forest	XGB Classifier	K-Nearest Neighbours	Support Vector Machine
Accuracy score (training):	0.7231	0.7138	0.9938	1.0000	1.000	0.9938
Accuracy score (test):	0.7105	0.7105	0.7105	0.7018	0.6930	0.7105
Precision score:	0.5048	0.5048	0.5048	0.5030	0.5012	0.5048
Recall score:	0.7105	0.7105	0.7105	0.7018	0.6930	0.7105
F1 score:	0.5903	0.5903	0.5903	0.5860	0.5817	0.5903
ROC AUC Score:	0.5000	0.5000	0.5000	0.4938	0.4877	0.5000
AUC from roc_curve:	0.5000	0.5000	0.5000	0.4938	0.4877	0.5000
Confusion Matrix	[[0 33] [0 81]]	[[0 33] [0 81]]	[[0 33] [0 81]]	[[0 33] [1 80]]	[[0 33] [2 79]]	[[0 33] [0 81]]

Model training with hyperparameter tuning:

- Logistic regression performed relatively well with 71.05% prediction of the test outcome and 72.31% prediction of y_train.
- Decision Tree achieved 71.05% prediction of test outcome with prediction of y_train at 71.38%.
- Random
- The predicted classes in the confusion matrix for all the models was distributed relatively the same.

Model for Production

The team chose Logistic Regression as the best performing model and hence model for production

CHALLENGES ENCOUNTERED

It has taken the team about 17 calendars days to accomplish this assignment. It has been a period of rigorous work and deep learning. All team members devoted all that it required to deliver this assignment. Key among the challenges encountered when handling the data are enumerated below:

- No header in the dataset: The team renamed the columns using the names provided in the project guideline
- Categorical datatype: The values in the gender columns were in text. Text values are not suitable for model learning, the team encoded these values into numerical values.
- Missing values in the dataset: Albumin and globulin ratio column had 4 null values. Upon further interrogations of the rows with the null values, the team observed no noticeable patterns hence concluded that the missing values were missing completely at random hence they replaced the values with mean of the column
- Different continuous columns exhibited outliers: The team performed log tranformation on the columns and then after that scaled the feature so that the wide variance in these columns was reduced make the models learn efficiently.
- Multiconearity in the data: some input columns exhibited high correlation between themselves. To make the models learnt efficiently,some of the columns were dropped and only important columns (features) were selected and used to train the models
- Imbalanced classes in the target variable: The team performed smote in order to address this challenge in the data